OpenFlex 导航开发指南

> 基于《OpenFlex 开源轮臂人形机器人使用说明书（高级篇）》第四章 "导航" 编写，同时关联建图（第二章）与定位（第三章）链路。 适用平台：ROS 2 Humble，工作空间 `~/openflex_all/openflex_ws` 。 本文聚焦 **可执行的开发与调试** ：怎么启动、怎么配参、怎么调优、出问题怎么排查。

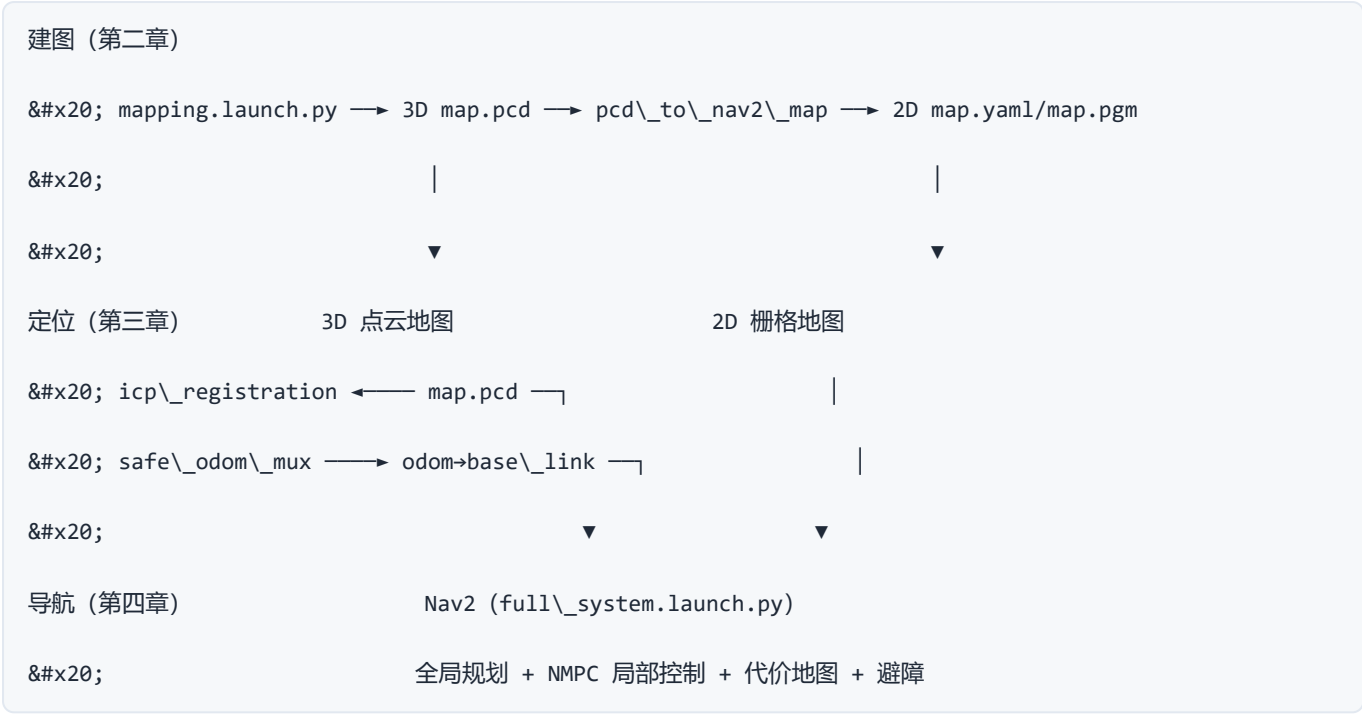
目录

- 一、导航系统总览
- 二、前置条件：地图与定位
- 三、快速开始
- 四、Nav2 架构与配置
- 五、NMPC 局部控制器（开发重点）
- 六、全局路径规划器与平滑器
- 七、代价地图配置
- 八、速度平滑器
- 九、行为服务器与恢复行为
- 十、实时避障输入：3D 点云转 2D 扫描
- 十一、导航调参与问题排查
- 十二、进阶开发方向

一、导航系统总览

1.1 三层链路关系

OpenFlex 导航 = **建图（离线）→ 定位（在线）→ 导航（在线）**，三者依赖关系如下：


建图（第二章）
  mapping.launch.py ⟶ 3D map.pcd ⟶ pcd_to_nav2_map ⟶ 2D map.yaml/map.pgm

定位（第三章）
3D 点云地图
2D 栅格地图
  icp_registration ⟵ map.pcd
  safe_odom_mux ⟶ odom→base_link

导航（第四章）
Nav2（full_system.launch.py）

全局规划 + NMPC 局部控制 + 代价地图 + 避障

1.2 导航数据流（运行时）

```
用户目标 Nav2 Goal

&#x20;       │

&#x20;       ▼

┌──────────────────────────────────────────────────────┐

│  BT Navigator（行为树决策）                            │

│    ├── Planner Server  ──► SmacPlanner2D 全局路径      │

│    ├── Controller Server ─► NMPC 局部路径跟踪          │

│    ├── Smoother Server ──► 路径平滑                    │

│    └── Behavior Server ──► spin / backup / wait 恢复   │

└──────────────────────────────────────────────────────┘

&#x20;       │ 速度指令

&#x20;       ▼

&#x20;  Velocity Smoother（加减速限制）

&#x20;       │ /cmd\_vel\_safe

&#x20;       ▼

&#x20;  底盘控制（swerve\_controller 舵轮运动学）

&#x20;       │

&#x20;       ▼

&#x20;  实际运动 → 里程计/点云反馈

避障输入：

&#x20; /fastlio2/body\_cloud ──► pointcloud\_to\_laserscan ──► /scan

&#x20;       ──► 局部代价地图障碍层 + 全局代价地图障碍层
```

1.3 定位与里程计供给（导航的前提）

数据 / 模块	作用	频率
`map.pcd`	ICP 全局定位用的 3D 点云地图	静态
`map.yaml`	Nav2 路径规划用的 2D 栅格地图	静态
`/fastlio2/lio_odom`	LIO/LIVO 短期里程计（LIVO 模式）	10 Hz

数据 / 模块	作用	频率
`/odom`	轮式里程计（底盘控制器输出）	50 Hz
`safe_odom_mux`	在 LIO 与轮式 odom 间安全切换，发布 `/odom_safe` 与 `odom→base_link`	50 Hz
`icp_registration`	实时点云与 3D 地图配准，发布 `map→odom`	\~100 Hz

最终导航使用的复合变换：`map → base_link = map → odom × odom → base_link`，兼顾高频平滑与全局准确。

二、前置条件：地图与定位

导航前必须完成建图并生成 2D 栅格地图（详见说明书第二章）。

2.1 建图（生成 3D 地图）

```
cd \~/openflex\_all/openflex\_ws

source install/setup.bash

export MAP\_NAME=your\_map

mkdir -p "\$HOME/MID\_360\_nav/openflex\_maps/3d/\$MAP\_NAME"

ros2 launch swerve\_navigation mapping.launch.py
```

建图时 RViz 关注 `/pgo/global_cloud` 与 `/fastlio2/body_cloud`。完成后保存地图：

```
ros2 service call /pgo/save\_maps interface/srv/SaveMaps \\

&#x20; "{file\_path: '\$HOME/MID\_360\_nav/openflex\_maps/3d/\$MAP\_NAME', save\_patches: true}"
```

2.2 3D 地图转 2D 导航地图

```
mkdir -p "\$HOME/MID\_360\_nav/openflex\_maps/2d/\$MAP\_NAME"

ros2 run pgo pcd\_to\_nav2\_map \\

&#x20; \--pcd "\$HOME/MID\_360\_nav/openflex\_maps/3d/\$MAP\_NAME/map.pcd" \\

&#x20; \--out-dir "\$HOME/MID\_360\_nav/openflex\_maps/2d/\$MAP\_NAME" \\

&#x20; \--poses "\$HOME/MID\_360\_nav/openflex\_maps/3d/\$MAP\_NAME/poses.txt" \\

&#x20; \--resolution 0.05 \\

&#x20; \--z-min 0.00 --z-max 2.30 \\

&#x20; \--obstacle-z-min 0.10 --obstacle-z-max 1.40 \\

&#x20; \--inflation-radius 0.10
```

输出 `map.yaml` + `map.pgm` 两个文件，即为导航输入。

地图效果调试速查：

现象	建议调整
地面被当成障碍	提高 `--obstacle-z-min`（如 0.10 → 0.15）
高处结构投影成障碍	降低 `--obstacle-z-max`
地图噪点多	增加 `--sor-mean-k 20 --sor-stddev 1.0`
地图太细碎	`--resolution` 放宽到 0.10
障碍物边界太贴近机器人	增大 `--inflation-radius`

2.3 定位准备

导航启动时会自动拉起定位链路（`icp_registration` + `safe_odom_mux`），无需单独启动。只需确认已生成：

- 3D 地图 `map.pcd`（含 `sc_data/` ScanContext 数据库，用于重定位）
- 2D 地图 `map.yaml`

三、快速开始

3.1 启动完整导航系统

```
export MAP\_NAME=your\_map

ros2 launch swerve\_navigation full\_system.launch.py \\

&#x20; pcd\_path:="\$HOME/MID\_360\_nav/openflex\_maps/3d/\$MAP\_NAME/map.pcd" \\

&#x20; map\_yaml:="\$HOME/MID\_360\_nav/openflex\_maps/2d/\$MAP\_NAME/map.yaml" \\

&#x20; initial\_pose:="\[0.0,0.0,0.0,0.0,0.0,0.0]" \\

&#x20; use\_rviz:=true
```

Launch 文件位置：
`src/openflex_chassis/navigation_layer/swerve_navigation/launch/full_system.launch.py`

3.2 常用启动参数

参数	默认值	说明
`pcd_path`	**必填**	ICP 定位用的 3D PCD 地图
`map_yaml`	**必填**	Nav2 用的 2D 栅格地图 YAML
`initial_pose`	`[0,0,0,0,0,0]`	ICP 初始位姿 `[x,y,z,roll,pitch,yaw]`
`use_rviz`	`true`	是否启动 RViz

参数	默认值	说明
`steering_can_interface`	`can5`	转向电机 CAN 接口
`driving_can_interface`	`can4`	驱动电机 CAN 接口
`nav_controller_max_wheel_speed`	`1.2`	导航模式轮速上限 (m/s)
`nav_controller_wheel_accel_limit`	`1.0`	导航模式轮速变化率上限 (m/s²)

3.3 RViz 操作流程

1. 确认地图已显示，机器人模型在地图附近；
2. 用 `2D Pose Estimate` 点击机器人实际位置和朝向（初始位姿不准时必做）；
3. 观察实时点云是否与地图重合（定位是否准确）；
4. 定位正确后，用 `Nav2 Goal` 点击目标位置；
5. 观察全局路径、局部路径和 `/cmd_vel_safe` 输出。

3.4 运行过程检查

```
ros2 run tf2\_ros tf2\_echo map base\_link     # 全局定位链路是否工作

ros2 topic hz /odom\_safe                     # 安全里程计

ros2 topic hz /scan                          # 实时避障输入

ros2 topic echo --once /cmd\_vel\_safe         # 实际速度指令
```

现象	判断
`map -> base_link` 持续发布	全局定位链路正常
`/scan` 有稳定频率	实时避障输入正常
`Nav2 Goal` 后出现路径并输出 `/cmd_vel_safe`	Nav2 控制链路正常

四、Nav2 架构与配置

配置文件：`src/openflex_chassis/navigation_layer/swerve_navigation/config/nav2_params.yaml`

4.1 组件列表

服务器	功能	插件
BT Navigator	行为树导航决策	Nav2 行为树节点集
Controller Server	局部路径跟踪	`nmpc_controller::NmpcController`
Planner Server	全局路径规划	`nav2_smac_planner/SmacPlanner2D`
Smoother Server	路径平滑	SimpleSmoother

服务器	功能	插件
Behavior Server	恢复行为	Spin, Backup, Wait
Velocity Smoother	速度平滑	VelocitySmoother
Waypoint Follower	航点跟随	WaypointFollower

4.2 导航核心参数

参数	值	说明
`global_frame`	map	全局参考帧
`robot_base_frame`	base_link	机器人帧
`odom_topic`	/odom_safe	安全里程计（必须确认该话题有数据）
`bt_loop_duration`	20 ms	行为树循环周期
`default_server_timeout`	20 s	服务器动作超时
`controller_frequency`	20.0 Hz	控制器执行频率

> 开发要点：
> `odom_topic`
> 指向
> `/odom_safe`
> 而非原始
> `/odom`
> 或
> `/fastlio2/lio_odom`
> ，这是三层定位架构的落地 ——Nav2 只认安全融合后的里程计。

五、NMPC 局部控制器（开发重点）

自定义 Nav2 Controller 插件 `nmpc_controller::NmpcController`，是 OpenFlex 导航区别于标准差速机器人的核心。

- 控制器代码：`src/openflex_chassis/navigation_layer/nmpc_controller/src/nmpc_controller.cpp`
- 求解器生成脚本：`src/openflex_chassis/navigation_layer/nmpc_controller/script/generate_solver.py`

5.1 模型与求解器

底盘被抽象为可全向平移的二阶非线性模型：

```
state   x = \[px, py, theta, vx, vy, omega]

control u = \[ax, ay, alpha]

dpx    = vx\*cos(theta) - vy\*sin(theta)

dpy    = vx\*sin(theta) + vy\*cos(theta)

dtheta = omega

dvx    = ax

dvy    = ay

domega = alpha
```

- 求解器：acados/CasADi，预测时域 **N=20, Tf=2.0s**
- 求解算法：SQP_RTI、HPIPM QP、ERK 积分、Gauss-Newton Hessian 近似
- 每个控制周期（20 Hz）只执行一次实时迭代（RTI）

有限时域优化目标（详见附录 A.8）：

```
minimize Σ ||x\_k - xref\_k||²\_Q + Σ ||u\_k||²\_R + Σ w\_c·costmap(xref\_k)² + ||x\_N - xref\_N||
²\_Qe

subject to: 状态方程、速度与加速度约束
```

5.2 优化权重表

权重	值	作用
`Q_px`, `Q_py`	65.0, 65.0	位置跟踪
`Q_theta`	8.0	航向跟踪
`Q_vx`, `Q_vy`, `Q_omega`	1.5, 0.8, 2.0	速度参考跟踪
`R_ax`, `R_ay`, `R_alpha`	0.08, 0.10, 0.12	控制加速度惩罚
`Q_e_px`, `Q_e_py`, `Q_e_theta`	85.0, 85.0, 12.0	终端位置 / 航向
`costmap_weight`	60.0	障碍代价权重

调参直觉： 提高 `Q_px/Q_py` → 路径跟随更紧密；提高 `costmap_weight` → 更保守绕障；提高 `R_*` → 运动更柔和。

5.3 速度限制

参数	值	说明
`vx_max`	0.50 m/s	最大前进速度
`vx_min`	-0.20 m/s	最大后退速度

参数	值	说明
`vy_max`	0.25 m/s	最大横向速度
`omega_max`	1.0 rad/s	最大角速度
`ax_max`	0.8 m/s²	前向加速度约束
`ay_max`	0.6 m/s²	横向加速度约束
`alpha_max`	1.8 rad/s²	角加速度约束

5.4 舵轮 native 路径跟踪策略

参数	值	说明
`allow_lateral_tracking`	true	允许全向平移
`swerve_native_mode`	true	车头朝终点方向，底盘横移贴路径
`start_heading_capture_dist`	0.80 m	起步捕获距离
`final_rotate_xy_tolerance`	0.18 m	终点原地旋转 XY 容差
`final_rotate_yaw_goal_tolerance`	0.08 rad	终点航向容差
`lio_odom_topic`	`/fastlio2/lio_odom`	LIO twist 初始化 NMPC 状态

> 这是四转四驱舵轮的关键优势：
> `swerve_native_mode=true`
> 时，车头始终朝向终点方向，通过底盘横移贴住路径，避免蛇形摆动。

5.5 局部 A* 避障（控制器内部补丁）

控制器内置短距离 A* 避障，局部路径前方出现障碍时快速绕行：

```
1\. 沿局部路径检查前方 local\_astar\_obstacle\_lookahead 范围内代价

2\. 连续路径点代价 >= local\_astar\_cost\_threshold → 判定阻塞

3\. 在 odom costmap 栅格上做 8 邻域 A\*

4\. 绕行路径平滑后拼回局部路径
```

参数	值	说明
`local_astar_enabled`	true	是否启用
`local_astar_cost_threshold`	0.40	触发绕行的代价阈值
`local_astar_reconnect_dist`	1.5 m	障碍后重连距离
`local_astar_obstacle_lookahead`	2.5 m	前方检查距离

5.6 目标容差

参数	值	说明
`xy_goal_tolerance`	0.18 m	Nav2 GoalChecker 位置容差
`yaw_goal_tolerance`	0.15 rad	Nav2 GoalChecker 航向容差
`required_movement_radius`	0.06 m	需移动的最小距离
`movement_time_allowance`	15.0 s	允许的最长不移动时间

六、全局路径规划器与平滑器

6.1 SmacPlanner2D

参数	值	说明
`plugin`	`nav2_smac_planner/SmacPlanner2D`	2D A* 规划器
`tolerance`	0.10 m	目标容差
`allow_unknown`	true	允许穿过未知区域
`max_iterations`	1000000	最大搜索迭代
`max_planning_time`	2.0 s	最大规划耗时
`cost_travel_multiplier`	10.0	代价地图权重

6.2 路径平滑器（SimpleSmoother）

参数	值	说明
`tolerance`	1.0e-10	收敛容差
`max_its`	1000	最大迭代次数
`do_refinement`	true	是否精化处理

七、代价地图配置

7.1 局部代价地图（Local Costmap）

参数	值	说明
`update_frequency`	10.0 Hz	更新频率
`publish_frequency`	5.0 Hz	发布频率
`width` / `height`	5 m / 5 m	地图尺寸
`resolution`	0.05 m	栅格分辨率

参数	值	说明
`rolling_window`	true	跟随机器人滚动
`global_frame`	odom	参考帧

机器人足迹（footprint）：

```
footprint: "\[\[-0.37,-0.31],\[-0.37,0.31],\[0.37,0.31],\[0.37,-0.31]]"
```

即 **0.74 m × 0.62 m** 的矩形（含安全裕量）。

障碍物层参数：

参数	值	说明
`observation_sources`	scan	数据源为 /scan
`max_obstacle_height`	2.0 m	最大障碍物高度
`obstacle_max_range`	4.5 m	标记障碍物最大距离
`raytrace_max_range`	5.0 m	射线清除最大距离

膨胀层参数：

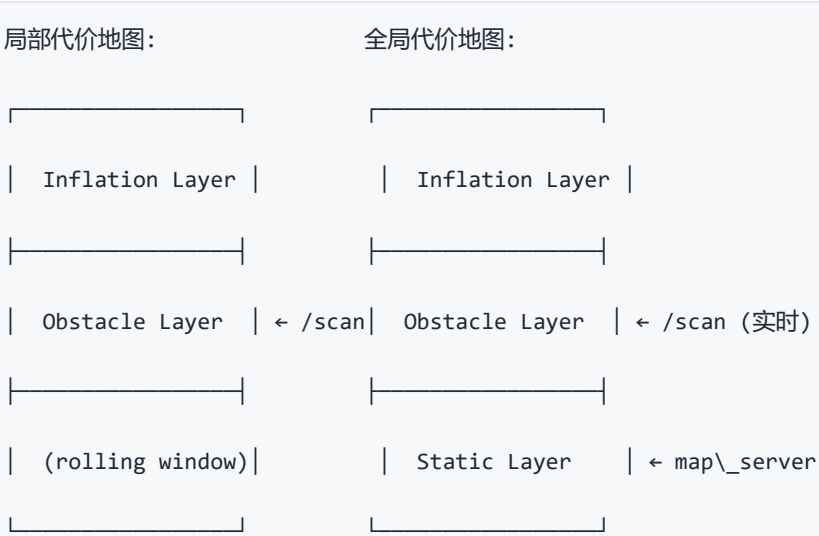
参数	值	说明
`inflation_radius`	0.30 m	障碍物膨胀半径
`cost_scaling_factor`	3.0	代价衰减速度

7.2 全局代价地图（Global Costmap）

参数	值	说明
`update_frequency`	5.0 Hz	更新频率
`publish_frequency`	2.0 Hz	发布频率
`resolution`	0.05 m	栅格分辨率
`global_frame`	map	参考帧
`track_unknown_space`	true	跟踪未知区域

全局代价地图额外包含 **Static Layer**（加载预生成 2D 地图）。

7.3 代价地图层叠结构



八、速度平滑器

对 NMPC 输出的速度命令做加速度 / 减速度限制，防止电机过载和底盘打滑。

参数	值	说明
`smoothing_frequency`	20.0 Hz	平滑频率
`scale_velocities`	true	超限时等比缩放速度
`feedback`	OPEN_LOOP	开环
`max_velocity`	[0.50, 0.22, 1.0]	[vx, vy, wz] 最大值
`min_velocity`	[-0.20, -0.22, -1.0]	[vx, vy, wz] 最小值
`max_accel`	[0.8, 0.6, 1.5]	最大加速度
`max_decel`	[-1.0, -0.6, -1.8]	最大减速度
`velocity_timeout`	1.0 s	速度命令超时自动停车

> 注意：NMPC 的 `vx_max=0.50` 与平滑器的 `max_velocity[0]=0.50` 需要保持一致，平滑器是最终执行上限。

九、行为服务器与恢复行为

导航遇到困难时，行为树触发恢复行为：

行为	说明
`spin`	原地旋转，清除周围障碍信息
`backup`	向后倒车
`drive_on_heading`	沿当前方向直行
`wait`	等待动态障碍物离开
`assisted_teleop`	辅助遥控

行为服务器参数：

参数	值	说明
`cycle_frequency`	10.0 Hz	行为执行频率
`simulate_ahead_time`	2.0 s	碰撞模拟前瞻时间
`max_rotational_vel`	1.0 rad/s	Spin 最大角速度
`min_rotational_vel`	0.4 rad/s	Spin 最小角速度

十、实时避障输入：3D 点云转 2D 扫描

`pointcloud_to_laserscan` 节点在导航运行时持续将 3D 点云投影为 2D 激光扫描，供代价地图使用。

10.1 数据流

```
/fastlio2/body\_cloud (PointCloud2, 3D)

&#x20;   |

&#x20;   └─(pointcloud\_qos\_relay)─► /livox/body\_cloud\_nav2

&#x20;                                |

&#x20;                                └─(pointcloud\_to\_laserscan)─► /scan (LaserScan, 2D)

&#x20;                                     |

&#x20;                                     └─► local\_costmap 障碍物层

&#x20;                                     └─► global\_costmap 障碍物层
```

10.2 参数

参数	导航模式值	说明
输入点云	`/livox/body_cloud_nav2`	经过 relay 自车滤波
`target_frame`	base_link	投影参考帧
`min_height`	-0.15 m	截取点云下界

参数	导航模式值	说明
`max_height`	0.40 m	截取点云上界
`angle_min` / `angle_max`	-π / π	扫描角度范围（360°）
`angle_increment`	0.0043 rad	角分辨率
`range_min`	0.45 m	最小有效距离
`range_max`	10.0 m	最大有效距离

10.3 算法流程

```
1\. 接收 3D 点云

2\. 变换到 target\_frame (base\_link)

3\. 按 min\_height/max\_height 过滤 Z 轴

4\. 按 angle\_increment 将点分入角度 bin

5\. 每个 bin 取最近的点

6\. 输出 2D LaserScan
```

> 开发要点：
> `min_height=-0.15 / max_height=0.40`
> 是机器人通过性高度窗口。若机器人经常与低矮障碍碰撞，需调高
> `max_height`
> 或调整 footprint。

十一、导航调参与问题排查

11.1 行为效果调参

现象	优先处理
机器人整体太快	降低 `desired_linear_vel`、Velocity Smoother 的 `max_velocity`
运动不够平稳	降低 `max_accel`、`max_decel`
路径跟随不紧密	提高 NMPC 权重 `Q_px`、`Q_py`
太靠近障碍物	增大 local costmap `inflation_radius`
窄通道过不去	减小 `inflation_radius`，检查 `footprint`

11.2 链路检查清单

按 "输入 → 处理 → 输出" 顺序排查：

层级	检查项	正常标准
传感器	`ping 192.168.1.173`	能收到 MID360 响应
传感器	`ros2 topic hz /livox/lidar`	\~10 Hz
传感器	`ros2 topic hz /livox/imu`	\~200 Hz
定位	`ros2 run tf2_ros tf2_echo map base_link`	持续有变换输出
定位	RViz 实时点云与地图重合	位置误差 < 20\~30 cm，朝向 < 10°
里程计	`ros2 topic hz /odom_safe`	50 Hz 稳定
避障输入	`ros2 topic hz /scan`	稳定频率，无大段空洞
控制输出	`ros2 topic echo --once /cmd_vel_safe`	有速度值且合理

11.3 常见故障速查

现象	优先处理
初始定位失败	用 RViz `2D Pose Estimate` 给更接近真实位置的初值
定位漂移或跳变	检查实时点云与 3D 地图是否重合
长走廊定位不稳	降低速度，让雷达看到几何特征
重定位识别失败	检查 `sc_data/` 目录是否存在
地图加载异常	确认 `map_yaml` 路径正确、`map.yaml` 与 `map.pgm` 在同一目录
机器人不动但无报错	检查 `odom_topic=/odom_safe` 是否有数据，`initial_pose` 是否给准
频繁触发恢复行为	膨胀半径过大或 footprint 偏大导致 "无路可走"

十二、进阶开发方向

12.1 定位精度保障

- ICP 持续重对齐：每 `continuous_realign_interval_sec`（5.0 s）以当前 `map→odom` 为初值执行 ICP 精配准，失败时保持上一次有效位姿，非阻塞执行。
- 长时间运行出现漂移时，优先检查实时点云与地图重合度，而非重启节点。

12.2 导航与 VLA 分层（推荐架构）

VLA 不建议直接学习长距离导航。推荐分层：

```
语言任务

&#x20; → 高层任务规划：选择目标区域和子任务

&#x20; → Nav2/NMPC：长程导航、避障、全局路径

&#x20; → VLA：到位后的底盘微调、升降、头部观察、双臂操作

&#x20; → ros2\_control：关节、电机、舵轮底层闭环
```

12.3 算法参考（导航相关论文）

方向	引用
全局路径规划 A*	Hart et al., 1968
NMPC 最优控制	Mayne et al., 2000；Verschueren et al., acados, 2022
栅格地图	Moravec & Elfes, 1985；Bresenham, 1965
舵轮运动学	Muir & Neuman, 1987；Campion et al., 1996
点云配准（ICP/NDT）	Besl & McKay, 1992；Biber & Strasser, 2003

12.4 关键文件索引

用途	路径
完整导航 Launch	`src/openflex_chassis/navigation_layer/swerve_navigation/launch/full_system.launch.py`
Nav2 参数	`src/openflex_chassis/navigation_layer/swerve_navigation/config/nav2_params.yaml`
NMPC 控制器	`src/openflex_chassis/navigation_layer/nmpc_controller/src/nmpc_controller.cpp`
NMPC 求解器生成	`src/openflex_chassis/navigation_layer/nmpc_controller/script/generate_solver.py`
安全里程计复用器	`src/openflex_chassis/navigation_layer/swerve_navigation/scripts/safe_odom_mux.py`
ICP 定位	`src/openflex_chassis/mapping_localization_layer/icp_registration/`
点云转激光	`src/openflex_chassis/navigation_layer/pointcloud_to_laserscan/`
3D 地图转 2D	`src/openflex_chassis/mapping_localization_layer/pgo/src/pcd_to_nav2_map.cpp`

本指南内容全部来源于《OpenFlex 开源轮臂人形机器人使用说明书（高级篇）》（传感器、建图、定位、导航、VLA 各章），命令与参数未经修改；实际部署时以工程内最新配置文件为准。